

4.1 System Architecture Overview

Udjat Track is a four-tier distributed system. The in-vehicle edge unit (Raspberry Pi 5) is responsible for all real-time sensor processing and on-device AI inference. The driver's Android phone hosts the application that displays alerts, manages the trip lifecycle, and uplinks summarised events to the cloud. A shared Spring Boot backend keeps fleet-manager state, trip history, and serves as the authoritative event store. A React web dashboard provides the fleet manager with a desktop-grade view of vehicles, drivers, trips, and real-time alerts across the fleet. The tiers communicate through three channels: a low-latency Wi-Fi WebSocket link between the Pi and the phone for live sensor and AI events; an HTTPS REST API between the phone and the backend for durable records; and a REST + WebSocket pair between the React dashboard and the backend for fleet-wide live monitoring.

4.1.1 Tier responsibilities

Table 11:

Tier	Responsibility	Implemented in
Edge IoT (Pi)	Sensor acquisition (IMU, ultrasonic, camera). Real-time driver-state classification. Crash, rollover, and proximity event detection. WebSocket broadcast to phone.	Python 3.10
Mobile (phone)	Driver UI: live trip map, alerts, emergency flows, trip lifecycle. Connects to Pi over WebSocket. Queues offline events. Uplinks to backend.	Kotlin + Jetpack Compose
Backend		
Web dashboard		

4.1.2 Communication and data flow

On trip start, the Android app authenticates against the backend, retrieves the trip definition (driver, vehicle, source/destination), and establishes a WebSocket connection to the Pi over the Pi-hosted Wi-Fi access point. The Pi streams 1 Hz telemetry frames and asynchronous event frames (CRASH, ROLLOVER, OBSTACLE_NEAR, driver-state transitions). The app aggregates these into driver-visible cards (fatigue, emergency, obstacle) and forwards qualifying events to the backend over HTTPS. The backend stores them and re-broadcasts them over its own WebSocket channel to all connected dashboard clients, so a fleet manager watching the React dashboard sees the alert appear in real time. The dashboard fetches fleet-wide state via REST on first load and subscribes to the backend WebSocket for subsequent live updates. When the trip ends the app posts a final summary; the Pi remains running so the next trip can resume immediately.

4.2 IoT Edge System (Raspberry Pi)

The edge unit performs all time-critical processing inside the vehicle. Three primary design goals shaped its implementation: (i) real-time response, events must be detected and broadcast within hundreds of milliseconds; (ii) operation independent of cellular or internet connectivity, so safety functions continue in tunnels and remote roads; and (iii) modest CPU/memory budget so the Pi can host the vision pipeline, sensor workers, and WebSocket server simultaneously without throttling.

4.2.1 Hardware platform

Table 12:

Component	Role
Raspberry Pi 5 (4 GB)	Main edge compute platform. Hosts the entire IoT, AI, and WS server.
Pi 5 active cooler	Sustained cooling for the vision pipeline running at near-100% on one core.
MPU-6050 IMU module	3-axis accelerometer + 3-axis gyroscope for crash and rollover detection.
6 × HC-SR04 ultrasonic sensors	Proximity sensors mounted front, rear, and two each on the left and right blind-spot positions.
Camera module (CSI/USB)	Driver-facing camera at 30 fps used as the vision-pipeline input.
12 V → 5 V step-down regulator	Power supply from the vehicle's 12 V electrical system.

4.2.2 Software stack

Python 3.10 was selected as the primary language for the edge unit for four reasons:

- Mature ecosystem for computer vision (MediaPipe, OpenCV) and numerical computing (NumPy), both of which are required by the AI pipeline.
- First-class Raspberry Pi support official GPIO libraries, hardware-acceleration bindings for the camera, and pre-built wheels for the major dependencies.
- Native asyncio and the websockets library, allowing a single-file async WebSocket server to coexist with synchronous sensor threads without manual event-loop plumbing.
- Rapid iteration, the detection thresholds (crash Δv , eye-closure timing, head-pose tilt) needed extensive empirical tuning during development, and Python's REPL/scripting model made each tuning cycle fast.

The principal libraries used on the Pi side are:

Table 13:

Library	Purpose
MediaPipe FaceMesh	Real-time facial landmark detection (468 landmarks per frame) for eye/mouth/pose features.
OpenCV (opencv-python)	Frame capture, image preprocessing, and drawing debug overlays.
NumPy	Vector and matrix operations used by the head-pose calculator and the IMU detector.
websockets (asyncio)	WebSocket server for live phone connectivity.
smbus2 / RPi.GPIO	Low-level I2C (IMU) and GPIO (ultrasonic trigger/echo) access.
pytest, pytest-cov	Unit-test framework and coverage tooling.
psutil	Resource profiling for the test in 5.1.6.

4.2.3 Module organization

The edge codebase is organized by sensor subsystem rather than by layer, so each sensor's acquisition, filtering, and detection logic lives in a single coherent directory:

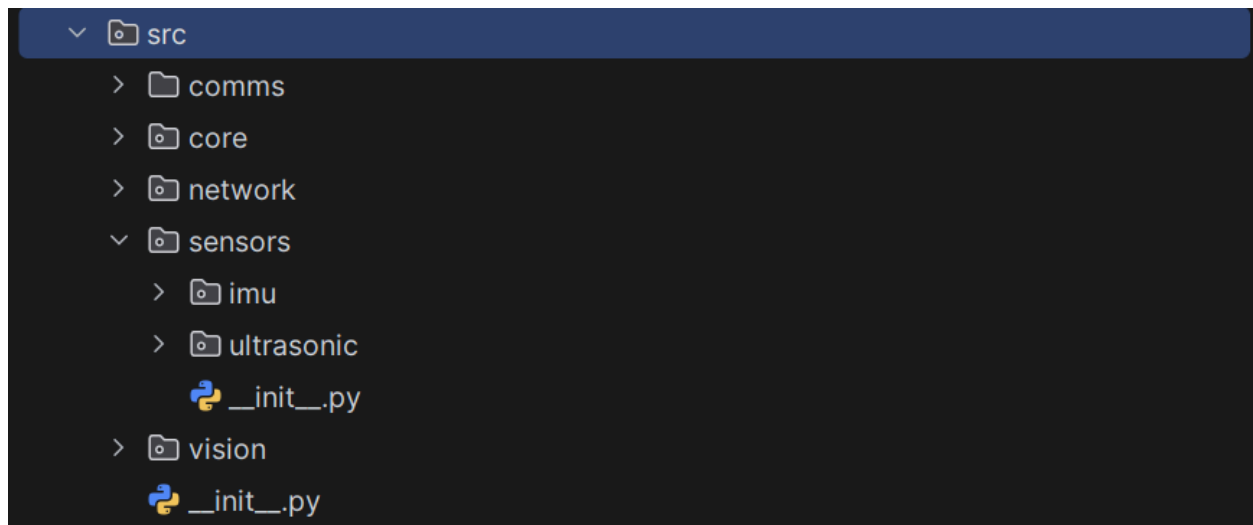


Figure 25: Top-level package structure of the Pi edge codebase.

Table 14:

Path	Contents
src/sensors/imu/	imu.py (hardware driver), worker.py (acquisition thread), event_detector.py (CRASH and ROLLOVER detection).
src/sensors/ultrasonic/	ultrasonic.py (driver), glitch.py (HC-SR04 blind-zone filter), worker.py, event_detector.py (OBSTACLE_NEAR).
src/vision/	main.py, worker.py, detection/face.py, detection/pose.py, state.py
src/comms/	ws_server.py (WebSocket broadcast server), injection_server.py (localhost CLI hook for demo events).
src/core/	shared_state.py (thread-safe state shared between workers), public_payload.py (frame schemas).
src/network/	pi_publisher.py (helper that wraps event publishing).

4.2.4 Threading model

The Pi runs five concurrent workers, each in its own thread, plus an asyncio event loop for the WebSocket server:

Table 15:

Worker	Rate	Output
Vision worker	~15 fps (camera)	Driver state, blink/yawn events into shared_state.vision
IMU worker	100 Hz	Acceleration + gyroscope into shared_state.imu
IMU event detector	100 Hz	CRASH and ROLLOVER events pushed to shared_state.events
Ultrasonic worker	20 Hz × 6 sensors	Filtered cm reading per sensor into shared_state.ultrasonic
Ultrasonic event detector	Polled at 20 Hz	OBSTACLE_NEAR events with sensor + position
WS server (asyncio)	1 Hz telemetry + on-event broadcast	JSON frames to all connected phones

All inter-thread communication goes through a single SharedState object protected by a lock, the event_detector workers read sensor snapshots, the WS server reads the latest snapshot for telemetry, and the events deque is drained by the WS event loop every 100 ms.

4.2.5 Detection algorithms

Three event detectors run in parallel on the Pi, each with a distinct algorithm:

Crash detection (FR-18, 3.6.5.1).

The IMU detector integrates total acceleration magnitude over a sliding window to compute a velocity change Δv . When Δv exceeds a calibrated threshold (~ 5 m/s of velocity change in under 200 ms) the detector enters a brief standby state to reject single-sample noise spikes; if the threshold is sustained or a second sample also exceeds the impact threshold, a CRASH event is emitted with the peak g-value and Δv as payload. A cooldown then disables re-firing for several seconds to prevent secondary impacts from generating duplicate events.

Rollover detection (FR-18).

Tilt is computed from the gravity vector using $\text{atan2}(a_y, a_z)$ for roll and $\text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$ for pitch. When the maximum tilt exceeds 45° a persistence timer starts; if the tilt remains above the threshold for at least 2 s, a ROLLOVER event is emitted. The persistence requirement prevents short-duration bumps and bank turns from triggering.

Proximity/obstacle detection (FR-32, 3.6.5.3).

Each HC-SR04 sample first passes through a blind-zone filter that detects the sensor's characteristic 400 cm spurious reading when an object is closer than ~ 2 cm and clamps it to 2 cm. The filtered distance is then compared against the 50 cm proximity threshold; if any sensor remains below threshold for at least 300 ms (debounce), an OBSTACLE_NEAR event is emitted with the sensor id and its mapped driver-facing position (FRONT, REAR, LEFT, RIGHT). A per-sensor cooldown of 5 s prevents continuous re-firing when an object remains alongside the vehicle.

4.3 Embedded AI and Computer-Vision Pipeline

The vision pipeline classifies the driver into one of four severity states, Alert, Drowsy, HighRisk, and Unconscious, at the native camera frame rate. All inference happens locally on the Pi; no video frame ever leaves the vehicle, which is both a privacy guarantee and a hard latency requirement (a cloud round-trip per frame would be incompatible with real-time alerting). The pipeline has three stages: facial landmark detection, per-frame feature extraction, and a temporal state classifier.

4.3.1 Why MediaPipe FaceMesh

MediaPipe FaceMesh was chosen as the upstream facial-landmark detector after evaluation against dlib's 68-landmark detector and a custom OpenCV Haar-cascade baseline. The decision factors were:

- Landmark density: FaceMesh provides 468 landmarks per face, including refined iris landmarks, supporting precise Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) calculations that the simpler detectors cannot match.
- On-device performance: TensorFlow Lite under the hood with GPU delegation where available; runs at ~15 fps on a Pi 5 vs ~3 fps for dlib's 68-landmark model.
- Robustness: handles partial occlusion (glasses, hand on face) and head-pose variation out to ~45° without retraining.
- Open-source and freely redistributable, which matters for fleet deployments where licensing fees would compound per vehicle.

4.3.2 Per-frame feature extraction

From the 468 landmarks, the pipeline derives four per-frame features that the temporal classifier consumes:

Table 16:

Feature	Computation
EAR (Eye Aspect Ratio)	Mean of the two eye-aspect ratios, each computed from six landmarks per eye as $(p_2 - p_6 + p_3 - p_5) / (2 \cdot p_1 - p_4)$. Closed eyes give $EAR \approx 0.10\text{--}0.16$; open eyes give $0.25\text{--}0.35$.
MAR (Mouth Aspect Ratio)	Vertical mouth distance divided by horizontal mouth width. Yawning gives $MAR \gtrsim 0.5$.
Head pose (roll, pitch, yaw)	Computed by SolvePnP against a canonical 3D face model using a subset of the FaceMesh landmarks. Used for unfocused/head-down detection.
Gaze direction	Iris-landmark offset from the eye centre, normalised by eye width. Used to detect off-road glance.

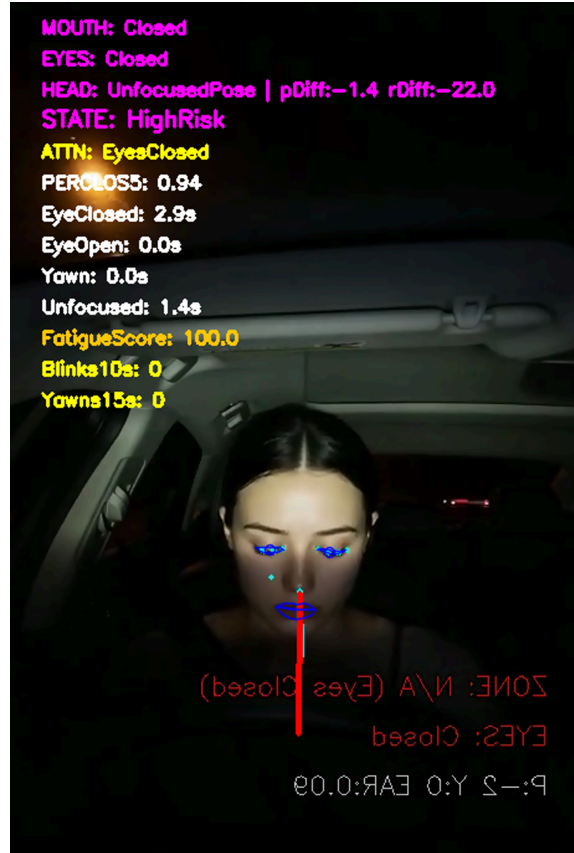


Figure 26: Driver-state monitoring window showing the extracted features

4.3.3 Temporal driver-state classifier

Per-frame features are noisy, and a single eye closure cannot distinguish a normal blink from a microsleep; temporal context is required. The classifier is implemented as a state machine over four severity states that accumulates evidence over time:

Table 17:

State	Entry criteria (production thresholds)
Alert	Default state. Eyes open ≥ 1.5 s, fatigue score $< \text{SCORE_DROWSY} - 8$.
Drowsy	Continuous eye closure ≥ 1.0 s OR sustained head-down ≥ 2.5 s OR fatigue score ≥ 45 OR severe-eye-evidence flag.
HighRisk	Continuous eye closure ≥ 3.0 s with fatigue score ≥ 70 , OR sustained low-EAR for ≥ 3.5 s with supporting PERCLOS5 ≥ 0.5 .
Unconscious	Continuous eye closure ≥ 4.0 s.

Three internal mechanisms make the classifier robust against detection noise:

- Fatigue score: a continuously updated 0–100 score that accumulates contributions from eye-closure duration, PERCLOS over the last 5 s, sustained low EAR, yawn rate, blink rate, and head-down duration; and decays when none of those is present. A behavioural cap prevents yawning + blinking alone from exceeding the HighRisk threshold.
- Minimum state-hold time: the classifier cannot change state more than once per second, absorbing single-frame jitter.
- Asymmetric recovery: descending the severity ladder requires longer eye-open intervals than ascending (1.5 s for Drowsy → Alert, 0.5 s for Unconscious → HighRisk), matching the asymmetry of human waking versus falling asleep.

4.3.4 Why the algorithmic state machine rather than an end-to-end model

An end-to-end deep model (e.g., a CNN+LSTM trained directly on raw frames) was considered and rejected for three reasons: (i) labelled fleet-driver footage at the scale required for end-to-end training was not available within the project; (ii) the explicit state machine produces interpretable decisions, when a Drowsy event fires, the system can explain which evidence caused it (eye closure, head pose, fatigue score) which a black-box model cannot; and (iii) thresholds can be tuned without retraining, allowing the model to be calibrated to individual cabin and camera setups in the field. The MediaPipe FaceMesh model itself is a deep neural network; the system is therefore a hybrid: a pretrained deep feature extractor followed by an interpretable rule-based classifier.

4.4 Mobile Application

The Android application is the driver-facing surface of Udjat Track. It runs on the driver's personal phone, connects to the Pi over the Pi-hosted Wi-Fi hotspot, and provides the live trip map, real-time alerts, emergency flows, and the bridge to the fleet-management backend. It is the only component of Udjat Track that drivers ever interact with directly, so usability, low-distraction visual design, and responsive alerts were primary design goals.

4.4.1 Programming language and framework

Kotlin is the primary application language; Jetpack Compose is the UI framework. The decision factors were:

- Kotlin is Google's recommended Android language and has first-class support across the Android Jetpack libraries used in the app (Navigation, Lifecycle, ViewModel, Hilt).
- Null safety in Kotlin eliminates an entire category of runtime crashes that were common in legacy Java Android code, important for a safety-related application that must remain stable throughout the trip.
- Jetpack Compose is declarative and lets the entire screen-update loop be driven by StateFlow emissions from view-models; this is a natural fit for the constant stream of WebSocket telemetry from the Pi where every frame updates the UI.
- Coroutines and Flow integrate cleanly with the underlying Pi WebSocket and the offline-sync queue, replacing the callback-style asynchronous code that would otherwise be needed.

4.4.2 Architecture

The application follows a layered (Clean-Architecture) package structure with MVVM at the UI layer and dependency injection via Hilt. Code is organised into four top-level packages under `com.example.udjattrack`, `core`, `data`, `domain`, and `feature`, sitting alongside the four root files that bootstrap the runtime (Figure 25). Each layer has a single responsibility, and the dependency rule runs strictly inward: `feature` depends on `domain` abstractions, `data` implements them, and `core` supplies cross-cutting infrastructure consumed by the others.

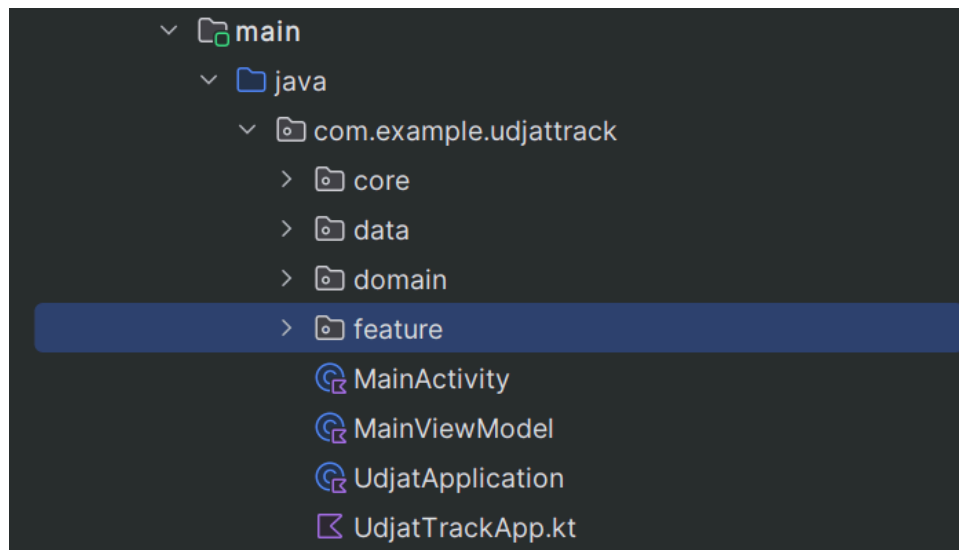


Figure 27: Top-level package structure of the Android module.

Table 18: Top-level layers

Layer	Purpose	Depends on
feature	Screens, view-models, and UI state for each user-facing feature.	domain, core
domain	Business rules: pure-Kotlin models, repository contracts, use-cases.	Nothing (pure Kotlin).
data	Concrete implementations of the domain contracts: Room, Retrofit, WebSocket, mappers, and offline sync.	domain, core
core	Cross-cutting infrastructure: DI graph, navigation, networking, services, audio, UI theme, utilities.	Nothing project-specific.

Core layer

The core package (Figure 26) holds infrastructure that is not specific to any one feature: the dependency-injection graph, navigation host, networking primitives, foreground services, audio playback, UI theme, and stateless helpers. Keeping these in a dedicated layer prevents cross-cutting concerns from leaking into feature code and gives every screen a single place to reach for shared utilities.

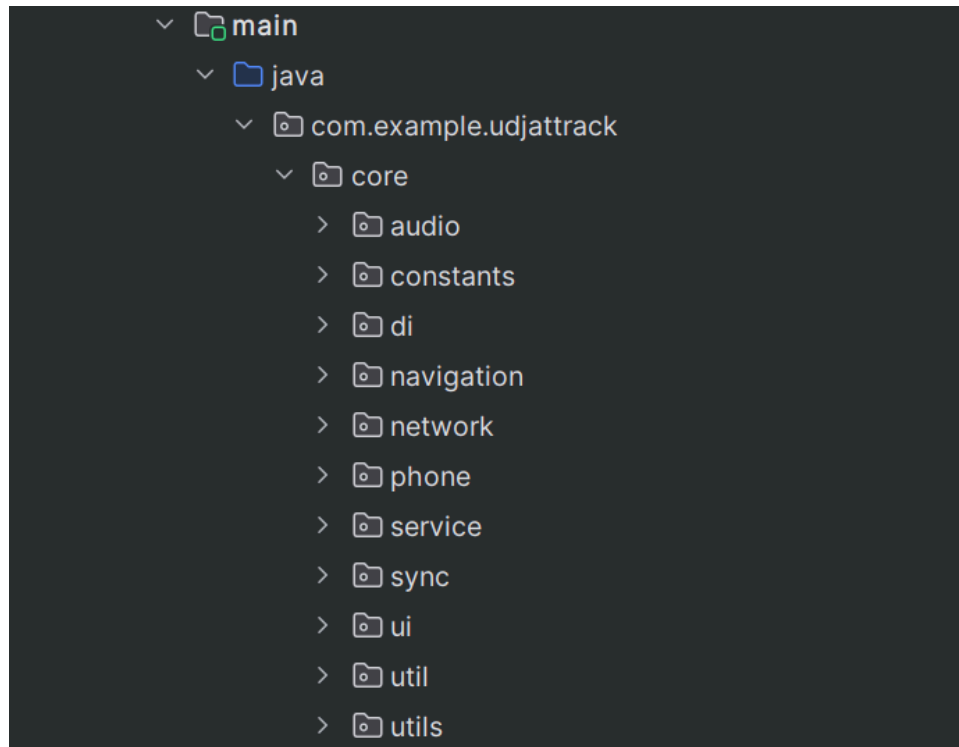


Figure 28: Subpackages of the core layer.

Table 19: Contents of core

Subpackage	Role
audio	AlertSounds : wraps ToneGenerator to play alarm-stream tones for safety-critical cards so they sound through Do-Not-Disturb.
constants	AppConstants: compile-time constants (e.g. BASE_URL).
di	Hilt modules (NetworkModule, AppModule, RepositoryModule, MapRepositoryModule, DispatcherModule) that bind data implementations to domain interfaces.
navigation	AppNavGraph: Compose-Navigation routes and arguments.
network	OkHttp/Retrofit interceptors and connectivity helpers shared by every HTTP client.
phone	rememberPhoneCaller: dialer-intent helper used by safety cards.
service	LocationClient: streams GPS fixes from a foreground location service while the app is in the background.
sync	Sync schedulers and constants backing the offline-sync flow in the data layer.
ui	Design-system primitives: UdjatTrackTheme, colour tokens (UdjatRed, UdjatGreen, UdjatLightBlue), reusable components (UdjatPrimaryButton).
util, utils	Stateless helpers: PolylineGeom, LocationUtils, formatters.

Data layer

The data package (Figure 27) provides concrete implementations of the contracts declared in domain. It is the only layer permitted to depend on Room, Retrofit, OkHttp, or DTOs, keeping persistence and transport details out of the rest of the codebase.

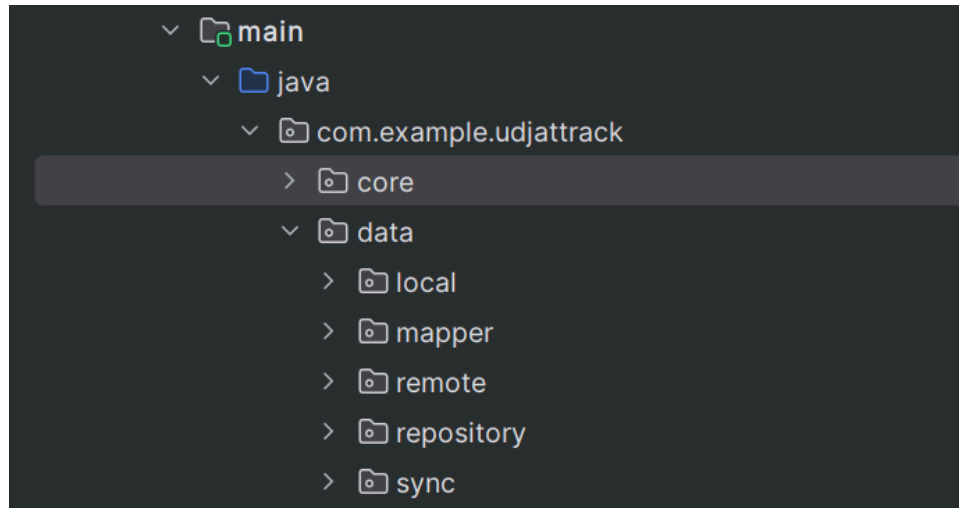


Figure 29: Subpackages of the data layer.

Table 20: Contents of data

Subpackage	Role
local	Room database, DAOs, and entity classes. Caches trips, the offline-event outbox, and session data.
mapper	DTO-to-domain and entity-to-domain converters; keeps JSON and database fields out of domain and feature.
remote	Retrofit services and DTOs for the backend; PiConnectionManager, PiWebSocketClient, and PiEventBus for the Pi WebSocket link (default ws://10.42.0.1:8765).
repository	Implementations of the domain repository interfaces (TripRepositoryImpl, RouteRepositoryImpl, ...), a glue that combines local, remote, and mapper.
sync	OfflineSyncManager which persists telemetry and event submissions to the Room outbox when offline and replays them when connectivity returns, ensuring no safety event is lost during a network drop.

Domain layer

The domain package (Figure 28) captures the business rules of the application as pure Kotlin. It contains no Android imports, no third-party framework dependencies, and no I/O, so every class can be compiled to plain JVM bytecode and unit-tested with JUnit alone, a property heavily exploited by the unit-test suite described in Chapter 5.

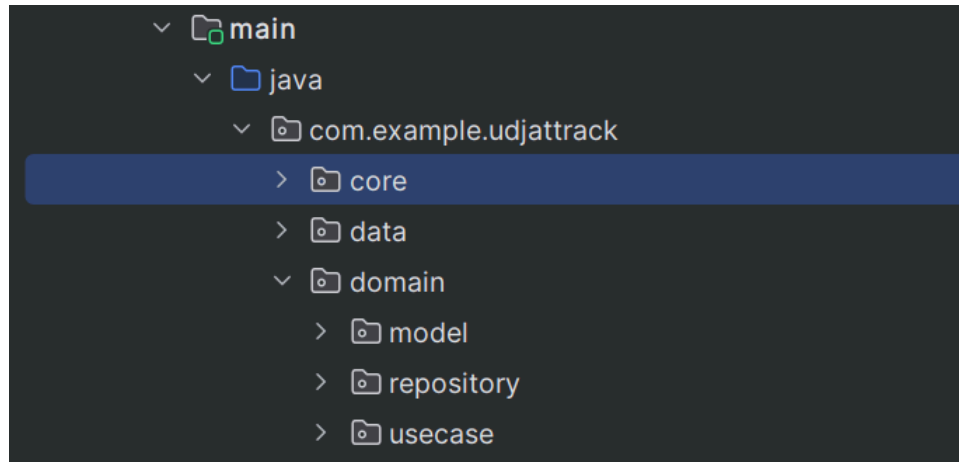


Figure 30: Subpackages of the domain layer.

Table 21: Contents of domain

Subpackage	Role
model	Pure Kotlin data classes : Trip, GeoPoint, EmergencyContact, DriverState, etc. These types form the vocabulary every other layer speaks in.
repository	Repository interfaces (TripRepository, RouteRepository, EmergencyContactRepository, ...) that the data layer fulfils. Use-cases depend on these interfaces, not concrete classes.
usecase	One class per single-purpose action: GetMyProfileUseCase, GetActiveTripsUseCase, SendSosUseCase, SendMaintenanceUseCase, SendIncidentUseCase, EndTripUseCase, AddEmergencyContactUseCase, GetEmergencyContactsUseCase, etc. View-models call use-cases, never repositories directly.

Feature layer

The feature package (Figure 29) holds the user-facing portion of the application. Each subpackage is one feature and follows the substructure `feature/<name>/ui/` (screens and feature-local Composables under `ui/components/`), `feature/<name>/viewmodel/`, and where useful `feature/<name>/state/` for the `UiState` data classes. A feature can be added, replaced, or removed without touching anything outside its own folder.

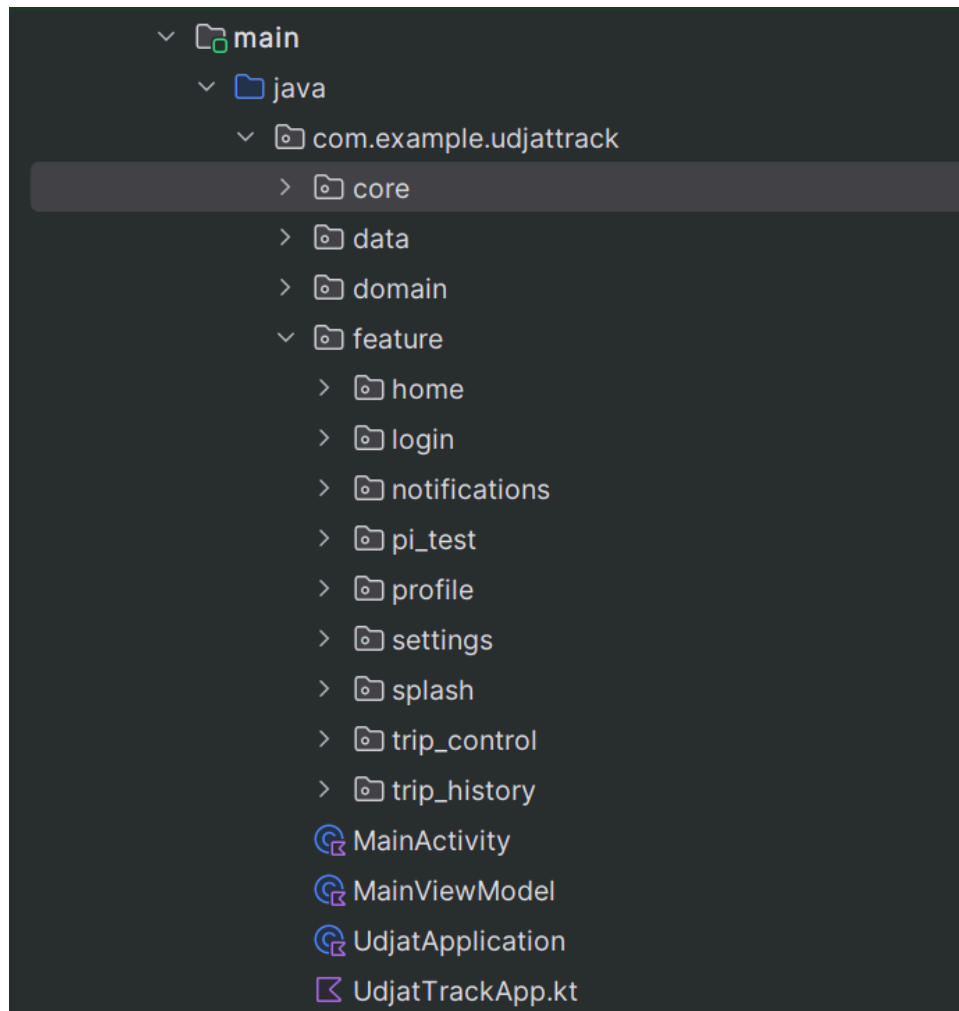


Figure 31: Feature subpackages.

Table 22: Feature Modules.

Feature	Purpose
splash	Launch screen: checks the persisted session and routes the user to login or home.
login	Authentication against the backend; persists the session on success.
home	Driver dashboard: shows today's trips, quick actions, and current vehicle.
trip_control	The active-trip experience: live map, speed/timer card, safety bottom-sheets (SOS, maintenance, incident, nearby help), and the safety-card overlays (FatigueDetectedCard, EmergencyHelpCard, CrashRolloverDetectedCard, AlertSentCard, ObstacleNearCard, TripOnBreakCard). The most stateful feature in the app and the primary consumer of Pi telemetry and events.
trip_history	Past trips with route playback, event timeline, and telemetry charts.
notifications	Push and in-app notification inbox.
profile	Driver profile and emergency-contacts management.
settings	App settings: language, theme, account, log-out.
pi_test	Diagnostic feature used during development and field testing: exposes the raw Pi WebSocket stream, connection status, last telemetry frame, and a manual reconnect button. Retained in the APK to simplify on-site debugging but not part of the production driver flow.

Runtime collaboration (MVVM, repositories, and DI)

The four layers above are *structural*. At runtime they cooperate in the well-known MVVM + repository pattern, wired together by Hilt: a Compose screen observes a `StateFlow<UiState>` exposed by a `@HiltViewModel`; the view-model dispatches user intents to one or more use cases; each use case talks to a repository interface declared in domain; the concrete repository in data calls the appropriate data source (Retrofit for the backend, the OkHttp WebSocket client for the Pi, Room for the local cache); and the DI graph in `core/di` binds every concrete implementation to its interface so that no upper layer holds a hard reference to a lower-layer class. Table 4.4.2-6 maps these MVVM roles back to the package layout described above.

Table 22: MVVM + repository roles and where they live in the package tree.

Role	Location	Contents
UI	feature/*/ui	Jetpack Compose composables organised by feature: home, login, splash, trip_control, trip_history, notifications, settings, profile, pi_test.
ViewModel	feature/*/viewmodel	StateFlow-based view-models that hold UI state, orchestrate use cases, and emit screen state. Examples: TripControlViewModel, HomeViewModel, LoginViewModel.
Use cases	domain/usecase/*	Single-responsibility coordinators for login, route resolution, trip creation, and event POSTing.
Repositories	data/repository	Implementations of domain repository interfaces. Caching, OSM / OSRM / Geoapify calls, backend POSTs, offline sync.
Data sources	data/remote, data/local	Retrofit APIs for the backend, OkHttp WebSocket client for the Pi, Room database for cache.
DI	core/di	Hilt modules that wire repositories, network clients, and use cases into view-models.

Because of this structure, swapping a data source (e.g. replacing the OSRM router with another provider, or pointing the Pi link at a different host) is a one-file change inside data, invisible to every view-model and screen.

4.4.3 Key libraries and tools

Library / tool	Purpose
Jetpack Compose + Material 3	UI rendering and theming.
Hilt	Dependency injection, scoping repositories and clients to the application.
Retrofit + OkHttp	HTTPS REST API client for the backend.
OkHttp WebSocket	Persistent connection to the Pi WebSocket server with built-in ping and reconnect handling.
Coil	Image loading for static-map previews (Geoapify static map).
Google Maps Compose	Live trip map; integration with Google's Maps SDK.
Google Play Services Location	Fused-location provider for GPS during the trip.
Room	Local persistence for notification history and offline-event queue.
Kotlin Coroutines + Flow	Asynchronous primitives across the codebase.
Turbine + MockK + JUnit	Unit testing of view-models, repositories, and DTOs.
Android Studio + Gradle (Kotlin DSL)	IDE and build system.

4.4.4 Feature overview and screens

The application prototype is organised into the following screens:

Screen	Purpose
Splash	Brief brand splash; initialises Hilt and checks the session token.
Login	Email / password authentication. On invalid credentials shows a translated error ("Wrong email or password") rather than the raw HTTP status.
Home	Driver's landing screen. Lists today's planned trips and previous trip cards with route preview (Geoapify static map of the OSRM-computed road shape).
Trip Start	Trip detail screen: shows driver name, vehicle, route, scheduled times. Driver presses "Start Trip" to begin.
Trip Control	The main in-trip dashboard. Live Google Map with current position snapped to the planned polyline; speed and trip-time indicators; Pi-connection status; safety button row (SOS, Maintenance, Incident, Nearby Help); event cards (Drowsy/HighRisk/Unconscious with 10 s self-cancel countdown, Obstacle direction card).

Screen	Purpose
Trip History	List of completed trips with start/end times, distance, and per-event summary.
Trip Details	Detail view of a single past trip including its planned route map.
Notifications	Notification centre: events received from the Pi and the backend, persisted with Room.
Profile	Driver profile, emergency contacts (add/edit), and account info.
Settings	App preferences.
Privacy	Privacy policy and data-handling summary.
Safety bottom sheets (Trip Control modals)	SOS bottom sheet (Send SOS / Call emergency contact / Back); Incident-report sheet (type, severity, description); Maintenance-report sheet (issue type, free-text notes); Nearby-help map (hospitals / gas stations / maintenance centres queried from OSM Overpass API).

4.4.5 Pi-connection management

PiConnectionManager is a Hilt-singleton that owns the OkHttp WebSocket client connecting to the Pi at ws://10.42.0.1:8765. Three properties of this component are notable:

- Single source of truth: telemetry, raw frames, last-known-state, and a typed event. SharedFlow are all owned by this manager, so any view-model (TripControl, debug overlay) can observe without independently managing the socket.
- Application-layer watchdog: independent of OkHttp's protocol-level ping, a coroutine fires every second and forces a reconnect if no frame has been received for more than 4 s, guarding against half-dead links.
- Per-frame latency instrumentation: every incoming frame computes (now – ts) and logs it as "lag <ms>" to logcat; this is the data source for the end-to-end latency benchmark in 5.1.4 of the testing chapter.

4.4.6 Live trip map and routing

Routing uses three independent OSM-derived services: Nominatim for geocoding (place name → coordinates), OSRM for actual road-network routing (start + end coordinates → polyline following real roads), and Geoapify for static-map previews. The TripControl screen renders the planned OSRM polyline on a Google Maps Compose map, and a snap-to-polyline algorithm projects each GPS fix onto the polyline so the marker rides the planned road smoothly, even when raw GPS is noisy. An off-route detector measures the distance from the snapped position to the polyline; if the distance exceeds 300 m for more than 5 s, the app fetches a fresh OSRM route from the current position and replaces the polyline.

4.5 Backend

4.6 Fleet-Manager Web Dashboard

4.7 Chapter Summary